

字符串操作

ab : a 和 b 连接

s^R : s 反转

$|s|$: s 的长度

语言操作

$L_1 L_2$: $\{ab \mid a \in L_1, b \in L_2\}$

L^n : $L^0 = \{\epsilon\}, L^n = \{s_1 s_2 \cdots s_n \mid s_1, s_2, \cdots, s_n \in L\}$

L^* : $\bigcup_{i=0}^{\infty} L^i$

L^+ : $\bigcup_{i=1}^{\infty} L^i$

$\bar{L}$: $\{s \in \Sigma^* \mid s \notin L\}$

L^R : $\{s^R \mid s \in L\}$

正则表达式

$\emptyset$: $L(\emptyset) = \emptyset$

ϵ : $L(\epsilon) = \{\epsilon\}$

$A|B$: $L(A|B) = L(A) \cup L(B)$

AB : $L(AB) = L(A)L(B)$

A^* : $L(A^*) = L(A)^*$

(A) : $L((A)) = L(A)$

DFA

定义: $M = (Q, \Sigma, \delta, q_0, F)$, 状态集合 Q , 字母表 Σ , 转移函数 $\delta \in Q^{Q \times \Sigma}$, 起始状态 $q_0 \in Q$, 终止状态集合 $F \subseteq Q$

拓展转移记号 $\delta^* \in Q^{Q \times \Sigma^*}$: 一次接受多个字符进行转移

接受的语言: $L(M) = \{s \in \Sigma^* \mid \delta^*(q_0, s) \in F\}$

NFA

定义: $M = (Q, \Sigma, \delta, q_0, F)$, 状态集合 Q , 字母表 Σ , 转移函数 $\delta \in (2^Q)^{Q \times (\Sigma \cup \epsilon)}$, ϵ 转移是不消耗字符的转移, 起始状态 $q_0 \in Q$, 终止状态集合 $F \subseteq Q$

同样也定义拓展转移记号, 一次接受多个字符进行转移, 得到可能到达的所有状态集合

接受的语言: $L(M) = \{s \in \Sigma^* \mid \delta^*(q_0, s) \cap F \neq \emptyset\}$

NFA到DFA

子集构造法: 对于NFA $M = (Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$, 定义一个DFA, 使其状态集合为 2^Q , 每个状态对应NFA中的一组状态, 转移定义为从当前这组状态出发接受给定字符后可能到达的所有状态集合, 起始状态定义为 $\{q_0\}$, 终止状态集合定义为 $\{S \in 2^Q \mid S \cap F \neq \emptyset\}$, 这一新DFA接受的语言和原NFA等同

由此也可以看出, DFA能接受的语言类和NFA等价, 称为正则语言RL

DFA最小化

对于两个状态 q_1, q_2 , 定义 k -等价为

$$\forall s \in \bigcup_{i=0}^k \Sigma^i, \delta^*(q_1, s) \in F \iff \delta^*(q_2, s) \in F$$

定义 ω -等价为对任意自然数 k 都 k -等价, 容易验证这些“等价”关系都确实是等价关系, 并且它们互相包含

DFA最小化的目标就是找到所有的 ω -等价类, 合并其中的状态

显然, 所有非终止状态是0-等价的, 所有终止状态也是0-等价的

两个状态 $k + 1$ -等价当且仅当 $\forall c \in \Sigma, \delta(q_1, c)$ 和 $\delta(q_2, c)$ 是 k -等价的

于是只需从0-等价开始迭代直到不动点，便找到了 ω -等价类，合并状态即可

DFA等价性判断

两个DFA等价当且仅当它们的起始状态 ω -等价

两个状态 ω -等价当且仅当 $\forall c \in \Sigma, \delta(q_1, c)$ 和 $\delta(q_2, c)$ 也是 ω -等价的

非终止状态和终止状态不可能 ω -等价。从起始状态出发，检查如果等价关系成立是否导出矛盾，即可确认两个DFA是否等价

正则语言性质

正则语言关于并集、交集（可用取反和并集构造）、连接、反转、取反、克莱尼星号封闭，具体构造略

每个正则语言都有对应的正则表达式，从DFA构造正则表达式的方法略

正则语言泵引理：考虑DFA的状态有限，足够长的字符串在DFA中必定经过重复状态。因此

$\forall L \in RL, \exists C \in \mathbb{N}, \forall s \in L \wedge |s| \geq C$, 存在一个分解 $s = xyz, |xy| \leq C$, 使得

$\forall i \in \mathbb{N}, xy^iz \in L$

泵引理可用于证明一个语言不是正则语言

二

上下文无关文法

定义：四元组 $G = (N, T, S, P)$, 非终结符集合 N , 终结符集合 $T, N \cap T = \emptyset$, 起始符号 $S \in N$, 产生式集合 P

每条产生式将一个非终结符展开为若干符号的序列，如 $A \rightarrow aAb$

派生： $xAy \Rightarrow xaAby$, 将字符串中的一个非终结符用某个产生式展开

连续派生： $\Rightarrow^*$, 多次派生的简记

最左派生： $\Rightarrow_{lm}$ ，每次都展开最左侧的非终结符

最右派生： $\Rightarrow_{rm}$ ，每次都展开最右侧的非终结符

推导树

CFG定义的语言是所有可以由起始符号派生得到的终结字符串，派生得到一个字符串的过程定义了一颗树，称为推导树

同一个字符串可能有不同的推导树，这称为二义性

一个CFG是否有二义性是不可判定的，而且有些语言存在内在二义性（不存在一个无二义性的CFG描述相同语言）

部分二义性可以使用优先级与结合性技巧消除

如 $E \rightarrow E + E \mid E \times E \mid a$ 有二义性，可改成 $T \rightarrow a \mid T \times a, E \rightarrow T \mid E + T$

NPDA

定义：七元组 $P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ ，状态集合 Q ，输入字母表 Σ ，栈上字母表 Γ ，转移函数 $\delta \in (2^{Q \times \Gamma^*})^{Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma}$ ，起始状态 $q_0 \in Q$ ，起始栈上符号 $z_0 \in \Gamma$ ，终止状态集合 $F \subseteq Q$

每次转移消耗一个栈顶符号，推入若干栈上符号（也可以加一个 ϵ 转移变成不一定消耗栈顶符号的形式，容易证明有没有 ϵ 转移都是等价的），在图上写成 $S, G \rightarrow G_1 G_2 \cdots G_n$ 形式

NPDA接受的语言与NFA类似，存在一条路径在终止状态停止即可（也可定义空栈为接受，二者是等价的，证明略）

每个CFG都有一个NPDA接受它定义的语言，只需在NPDA的栈上展开符号即可，具体略

每个NPDA都有一个对应的CFG，方法和CFG到NPDA类似，方向反转即可，具体略

因此CFG和NPDA定义了同样的语言，称为上下文无关语言CFL

NPDA比DPDA更强

CFL

CFL关于并集、连接、反转、克莱尼星号封闭，关于交集不封闭（ $a^n b^n c^m$ 和 $a^n b^m c^m$ 是CFL，而 $a^n b^n c^n$ 不是CFL），于是关于取反、做差也不封闭

CFL泵引理：考虑一个CFL中非终结符的数量是有限的，于是对于足够长的字符串，它的推导树深度足够大，那么一定存在一条从根出发的路径上会出现重复的非终结符，可以用更大的这颗子树替换更小的子树得到另一个字符串。因此 $\forall L \in CFL, \exists C \in \mathbb{N}, \forall s \in L \wedge |s| \geq C$, 存在一个分解 $s = uvwxy, vx \neq \epsilon, |vwx| \leq C$, 使得 $\forall i \in \mathbb{N}, uv^i wx^i y \in L$

自顶向下解析

自顶向下解析：从起始符号开始逐步展开，直到展开到给定字符串

左递归消除：自顶向下解析器难以处理左递归（一个非终结符经过若干步展开后最左侧的非终结符是自己），需要使用技巧消除左递归

直接的左递归可以改写成开头加增量的形式，如 $E \rightarrow Ea \mid b$ 可以改写为 $E \rightarrow bE', E' \rightarrow \epsilon \mid aE'$ ；间接的左递归可以展开到直接的左递归

LL(1)解析器：每次考虑最左侧的符号，消耗终结符，展开非终结符。LL(1)代表只需要提前看一个字符，就可以确定展开最左侧的非终结符使用的产生式，这使用预测分析表进行（非终结符与一个提前看符号组成的二维表 M ）

LL(1)解析器的构建使用FIRST和FOLLOW进行，FIRST是一个符号展开后所有可能的第一个终结符组成的集合（可能为空串则有 ϵ ），FOLLOW是紧邻一个符号之后可能出现的所有终结符组成的集合（起始符号的FOLLOW有 $\$$ ，即字符串结尾），二者可以通过迭代传播求出，具体略

然后对于规则 $A \rightarrow \alpha, \forall a \in FIRST(\alpha), M(A, a) = A \rightarrow \alpha$ ；若 $\epsilon \in FIRST(\alpha)$ 则还有 $\forall a \in FOLLOW(\alpha), M(A, a) = A \rightarrow \alpha$

自底向上解析

自底向上解析：基于栈进行，每次向栈内移入一个新符号，或者对栈内符号进行规约（折叠产生式），直到将整个给定字符串规约为起始符号就说明解析成功

LR(0)解析器：不断向栈内移入符号，直到看到某个产生式的右手侧就进行规约。检查是否看到了产生式的右手侧可以用DFA进行

语义分析

在推导树上进行类型检查和传递

三

三地址码

三地址码：每条指令包含至多三个地址

如 $x = y \text{ op } z$, $\text{goto } L$, $\text{if } x \text{ op } y \text{ goto } L \text{ else goto } L'$ 等

语法制导定义

在产生式上附加计算规则，通过语法来定义属性的计算方式

用于计算属性、语义检查、语法制导翻译等，具体没什么好说的

SSA

SSA：每个变量只有一个定义，使用 ϕ 函数合并来自不同前驱的定义，可以保证清晰的 def-use 链

基本块：只有单一入口和出口的一段连续指令

转换到SSA

支配：如果从程序入口到基本块 A 的任何一条路径都经过基本块 B ，称 B 支配 A

后支配：如果从基本块 A 到程序出口的任何一条路径都经过基本块 B ，称 B 后支配 A

严格(后)支配：若 B (后)支配 A ，且 $B \neq A$ ，则 B 严格(后)支配 A

直接支配：若 B 严格支配 A ，且不存在 $C \neq A, B$ 使得 B 支配 C ， C 支配 A ，称 B 直接支配 A

基本块的支配关系构成一颗有根树，每个节点的父节点是它的直接支配者

支配边界：基本块 B 的支配边界是由所有这样的 A 组成的集合： A 有至少一个前驱被 B 支配，但 A 不被 B 支配。支配边界是刚刚离开 B 的支配范围的基本块集合

迭代支配边界是在基本块集合 $\mathbb{B}$ 上不断求支配边界加入其中，迭代到不动点的结果

对于一个变量，它的所有定义组成的集合的迭代支配边界就是需要插入 ϕ 函数合并定义的地方，可以证明这样插入的 ϕ 总数是最小的（尽管有些合并的结果不会被用到）

四

目标代码生成

一堆体系结构知识略

寻址模式

以下 $*R2$ 代表寄存器 $R2$ 中的值

LD $R1, a(R2)$: 取 $(*R2)+a$ 地址处的数据放入 $R1$

LD $R1, 100(R2)$: 取 $(*R2)+100$ 地址处的数据放入 $R1$

LD $R1, *R2$: 取 $*R2$ 地址处的数据放入 $R1$

LD $R1, *100(R2)$: 取 $(*R2)+100$ 地址处的数据作为新地址，取该处的数据，放入 $R1$

LD $R1, \#100$: 将字面量100放入 $R1$

块内优化

局部公共子表达式：将不同部分的公共表达式合并为一次计算

强度削弱：将部分算术操作替换为等价的简单操作，如

$$x^2 = x * x, x * 2 = x + x, x/2 = x * 0.5$$

使用机器习语：如 $x = x + 1$ 可以直接编译到INC x

去除冗余的LOAD和STORE

窥孔优化：匹配一小窗口内特定的指令序列，替换成更高效的序列

转换出SSA*

从SSA生成机器代码需要移除 ϕ 函数，我们希望在保证程序等价性的条件下尽量将 ϕ 函数中的多个变量直接合并为一个

对于 $a = \phi(a_1, a_2, \dots, a_n)$ ，消除的方法如下：

1. 在每个 a_i 对应的基本块末尾插入 $a'_i = a_i$
2. 将原 ϕ 函数替换为 $a' = \phi(a'_1, a'_2, \dots, a'_n)$
3. 在替换后的 ϕ 函数之后插入 $a = a'$
4. 将所有 a' 和 a'_i 替换为一个相同的新变量
5. 通过向前驱的末尾插入赋值语句的方式，消除替换后的 ϕ 函数
6. 上一步插入的赋值是并行进行的，需要将其串行化
7. 消除多余的赋值

寄存器分配

虚拟寄存器是无限的，而实际的寄存器是有限的，需要决定何时将哪些量留在物理寄存器中，哪些放在栈上

Speed: Registers > Memory

Physical machines have limited number of registers

Register allocation: ∞ virtual registers $\rightarrow k$ physical registers

Requirement:

Produce correct code using k or fewer registers

Minimize loads, stores, and space to hold spilled values

Efficient register allocation ($O(n)$ or $O(n \log n)$)

局部寄存器分配

在基本块内部没有跳转，我们可以轻易地求出块内变量的存活区间

MAXLIVE：对于一个语句，它结束时还有多少变量存活

当有语句的MAXLIVE超过实际寄存器数量时，在这条语句开始之前需要把一个量spill到栈上，选择spill的变量的方法是选择距其下一次使用最远的变量

全局寄存器分配

全程序的寄存器分配一般使用冲突图进行

如果两个变量的存活区间有交，我们称它们是冲突的，在冲突图上二者之间有一条无向边

最终我们需要spill掉一些变量使得冲突图顶点是可以被 k 染色的，其中 k 是可用的物理寄存器数量

图染色近似算法

Chaitin算法：度小于 k 的顶点最后总是可以被染色，每次把这样的点从图中删掉放入栈中，对剩下的顶点继续过程，直到图为空就可以以删除顺序的逆序（出栈序）依次染色节点。如果过程中图的每个点度数都大于等于 k ，选择一个顶点spill掉再继续。

Chaitin-Briggs算法：考虑到度大于等于 k 的点实际上也可能可以被染色，相比于Chaitin算法，这个算法在所有点度数都过大时不直接spill掉点，而是仍然放入栈中。最后出栈染色的时候再检查是否能够染色，如果不能再在这时spill。

五

指令调度

由于CPU实际上有并行地执行多个指令的能力，编译时如果将独立的、可并行的若干指令放在一起，可以让CPU并行地执行这些指令，提高程序执行的效率

Hardware only can execute instructions that have been fetched

Hardware has limited space to buffer stalled operations

Compilers can place independent operations close together to allow better hardware utilization

块内调度

读与读：无依赖，可以任意调换

写后读：真依赖，不能调换顺序

读后写：假依赖（存储依赖），分配额外的寄存器可以消除

未消除的依赖构成依赖图，为一DAG，能以该图的任意拓扑序重新调度指令

列表调度：考虑依赖图的拓扑序，对每个时钟周期尝试安排已就绪的指令，若有多个就绪的指令，以特定优先级来确定安排哪一个（如优先安排路径延迟最高的等等）

跨块调度*

拓展基本块（EBB）：一组基本块，满足这组基本块只有一个入口，同时除入口块之外的其他基本块只有一个前驱

对于一个拓展基本块内的的一条路径，指令调度和块内调度方法一致，不过在指令移动时需要注意，向上/向下移动时若不满足对应的支配关系要进行额外处理

每次选择一条EBB路径做指令调度，然后把这个路径从图里删去，重复这样做直到图为空

对循环的处理*

可以考虑展开循环后调度，也可以考虑循环整体来优化，不过没有完美的方法

六

数据流分析

基本结构：数据流分析的结果组织为facts，每个基本块的入口和出口分别有in facts和out facts，每个基本块描述一个transfer function，用于从in/out facts产生out/in facts，同时还有一个meet function用来合并两个facts的结果，用于合并基本块前驱/后继传递的结果

以前向数据流分析为例，每个基本块的in facts为其所有前驱的out facts的合并，out facts则是将自身的transfer function应用于in facts的结果。从某个初始条件开始迭代，满足这些条件的最小/最大不动点就是数据流分析需要的结果。

Worklist算法：可以发现每个基本块的in/out facts只与前驱的out facts/后继的in facts有关，而out/in facts是由in/out facts通过transfer function求出，因此只有某一前驱/后继的facts更新时

这个基本块才需要更新。在每次更新后将当前基本块的后继/前驱加入列表，只需不断从列表中取出基本块更新即可，而无需更新所有的基本块。

典型的数据流分析：可达定值分析、可用表达式分析、活跃变量分析，三个分析的转移函数全都是 gen-kill 标准型，即 $D' = (D - Kill) \cup Gen$

可达定值分析：哪些定义可能不被覆盖地到达当前点。前向分析，meet function 是并，facts 初始化为空集，kill 被覆盖的定义，gen 当前基本块中的定义。

可用表达式分析：哪些表达式在到达当前点之前一定被计算过，而且使用到的变量值没有被改变。前向分析，meet function 是交，facts 初始化为全集，entry out facts 初始化为空集，kill 包含当前块中被重新定义的变量的表达式，gen 当前基本块中被计算的表达式。

活跃变量分析：哪些变量的定义在当前点之后还可能被使用。后向分析，meet function 是并，facts 初始化为空集，kill 当前基本块中定义的变量，gen 当前基本块中使用的变量。

还有许多不特别的分析，还有不甚特别的稀疏分析技术，不在考察范围内，略

符号执行

三个重要的敏感性：路径敏感（特定路径）、流敏感（特定程序点）、上下文敏感（特定上下文）

符号执行提供路径敏感，但是带来路径爆炸问题，对外部函数也难以处理。对断言等的检查还引入约束求解的需求。

路径爆炸可以通过 State Merging（合并不同路径的状态）、Loop Induction（对循环效果做归纳）缓解，对外部函数的处理可以用 Concolic Execution（结合符号执行和具体执行的值）进行

布尔可满足性

CNF：形如 $\bigwedge_i \bigvee_j l_{i,j}$ 的公式

可满足性：称公式 F 是可满足的，当且仅当存在一种对于 F 中所有布尔变量的指派，使得最后 F 的值为真

同时可满足性：对于公式 F_1, F_2 ，如果 F_1 可满足当且仅当 F_2 可满足，称它们是同时可满足的

Tseitin变换：对于任何公式 F ，存在一个CNF公式 F_c ，满足 F 与 F_c 同时可满足，且 F_c 至多比 F 长常数倍。具体方法为考虑 F 的语法树，将每个节点都对应到一个新变量，用于描述这个节点的语义

如对于 $A \Rightarrow (B \wedge C)$ ，可将其变换为 $x_1 \wedge (x_1 \iff (A \Rightarrow x_2)) \wedge (x_2 \iff B \wedge C)$ ，再将其中的 $\iff$ 等变换到使用 $\neg$ 和 $\vee$ 的形式即可

DPLL：每次选择公式中的一个布尔变量，赋真或假值，并在公式中传播该值，当公式化简到假时回滚重试，重复直到找到一个解或者尝试完所有可能

CDCL：每次出错时从矛盾式中学习不可能的赋值组合，使得DPLL更快

可满足性模理论

对于一阶谓词逻辑，考虑谓词的输入，如果可以建模为位向量就可以使用以下方式求解：

1. 限制谓词输入到固定字长
2. 将固定字长的位向量拆成若干单独位，并将对应的操作也转换为对单独位的操作，将一阶谓词逻辑公式转换为命题逻辑公式
3. 使用DPLL/CDCL求解

七

跨过程分析*

主要是提供上下文敏感性，方案众多

有在经典的数据流分析基础上设计的，基于克隆的方法和基于函数概要的方法

还有对于满足分配律的数据流分析，建模到CFL可达性的方法，在此之上又有许多优化方法

内容复杂，不在考察范围中，不介绍

指针分析

Andersen算法：

语句	对应约束	简记
$y = \&x$	$x \in \text{pts}(y)$	
$y = x$	$\text{pts}(x) \subseteq \text{pts}(y)$	
$*y = x$	$\forall v \in \text{pts}(y), \text{pts}(x) \subseteq \text{pts}(v)$	$\text{pts}(x) \subseteq \text{pts}(*y)$
$y = *x$	$\forall v \in \text{pts}(x), \text{pts}(v) \subseteq \text{pts}(y)$	$\text{pts}(*x) \subseteq \text{pts}(y)$

Steensgaard算法：

语句	对应约束	简记
$y = \&x$	$x \in \text{pts}(y)$	
$y = x$	$\text{pts}(x) = \text{pts}(y)$	
$*y = x$	$\forall v \in \text{pts}(y), \text{pts}(x) = \text{pts}(v)$	$\text{pts}(x) = \text{pts}(*y)$
$y = *x$	$\forall v \in \text{pts}(x), \text{pts}(v) = \text{pts}(y)$	$\text{pts}(*x) = \text{pts}(y)$

前者的实现基于图闭包（指针流图），后者将包含改为相等，可以用并查集实现

二者都是可能分析，是实际的指向关系的上近似，后者效率更高，但是精确度更低

基于Datalog的数据流分析

Datalog是关于facts和rules的语言

rules说明能从什么条件推出什么结论，定义语法是 $P(a,b,...) \text{ :- } A(...), B(...), \dots$

facts则是从外部加入的事实，用于推导其他结论

使用Datalog进行数据流分析，只需要定义数据流分析的规则，而不再需要自己解决产生结果的过程，更为方便

如使用Datalog实现可达定值分析，用到的谓词如下：

def(B, N, X)：基本块B中的第N条语句可能定义变量X

$\text{succ}(B, N, C)$: 基本块C是B的后继, B有N条语句

$\text{rd}(B, N, C, M, X)$: 基本块C的第M条语句对变量X的定义可能到达基本块B的第N条语句

定义规则如下:

$\text{rd}(B, N, B, N, X) :- \text{def}(B, N, X)$

$\text{rd}(B, N, C, M, X) :- \text{rd}(B, N-1, C, M, X), \text{def}(B, N, Y), X \neq Y$

$\text{rd}(B, 0, C, M, X) :- \text{rd}(D, N, C, M, X), \text{succ}(D, N, B)$

即可自动进行可达定值分析, 指针分析等也类似, 不赘述
